

TrustPass Team Project Plan

TrustPass — Team Project Plan

Passwordless Authentication for Bank Logins

Domain: Full-Stack Web Development

Team Size: 4 Members

Version: 1.0

Date: August 2026

Table of Contents

1. Executive Summary
2. Problem Statement & Solution
3. Why Your Last Project Failed to Integrate
4. Architecture Decision — Single Integrated App
5. Team Structure & Role Assignments
6. Member 1 — Integration Lead & Database Owner
7. Member 2 — Retail Bank Passkey Authentication
8. Member 3 — Commercial Bank Hardware Key Authentication
9. Member 4 — UI/UX, Design System & Demo Polish
10. Shared Repository Structure
11. Git Workflow & Integration Rules
12. Environment & Tooling Setup (All Members)
13. Database Schema & API Contracts
14. 4-Week Sprint Timeline
15. Daily Integration Checklist
16. Judging Criteria — How We Score Points
17. Innovation & Bonus Features Backlog
18. Demo Script (5 Minutes)
19. Risk Register & Mitigation
20. Originality & Plagiarism Guidelines
21. Pre-Submission Checklist

1. Executive Summary

TrustPass is a passwordless banking platform where users register once, enroll a passkey, then choose between two bank portals — each with a different authentication method:

Portal	Target User	Auth Method
TrustPass Retail Bank	Individuals & families	Biometric passkeys (Touch ID / Face ID / Windows Hello)
TrustPass Wealth & Commercial	Businesses & HNW clients	FIDO2 hardware security keys (YubiKey)

Core principle for this team: Nobody builds an isolated frontend, backend, or API in a separate repo. Everyone works in **one monorepo**, on **one running app**, merging to `main` daily.

Current project status (already built): - Next.js 16 registration UI - Supabase (PostgreSQL) account storage - Passkey enrollment simulation modal - Two-bank dashboard at `/dashboard`

Remaining work: Real WebAuthn flows for both banks, login pages, session management, UI polish, and demo readiness.

2. Problem Statement & Solution

Problem

Traditional bank logins rely on passwords that are phishable, reused, and leaked in breaches. Banks need a secure, user-friendly alternative.

Our Solution

A unified TrustPass identity layer that: 1. Registers the user with verified profile data (stored in PostgreSQL via Supabase) 2. Enrolls a device-bound passkey (WebAuthn / FIDO2) 3. Routes the user to the correct bank portal based on account type 4. Authenticates using **different passwordless methods per bank** — demonstrating flexibility for retail vs. commercial use cases

Tech Stack (Mandatory + Chosen)

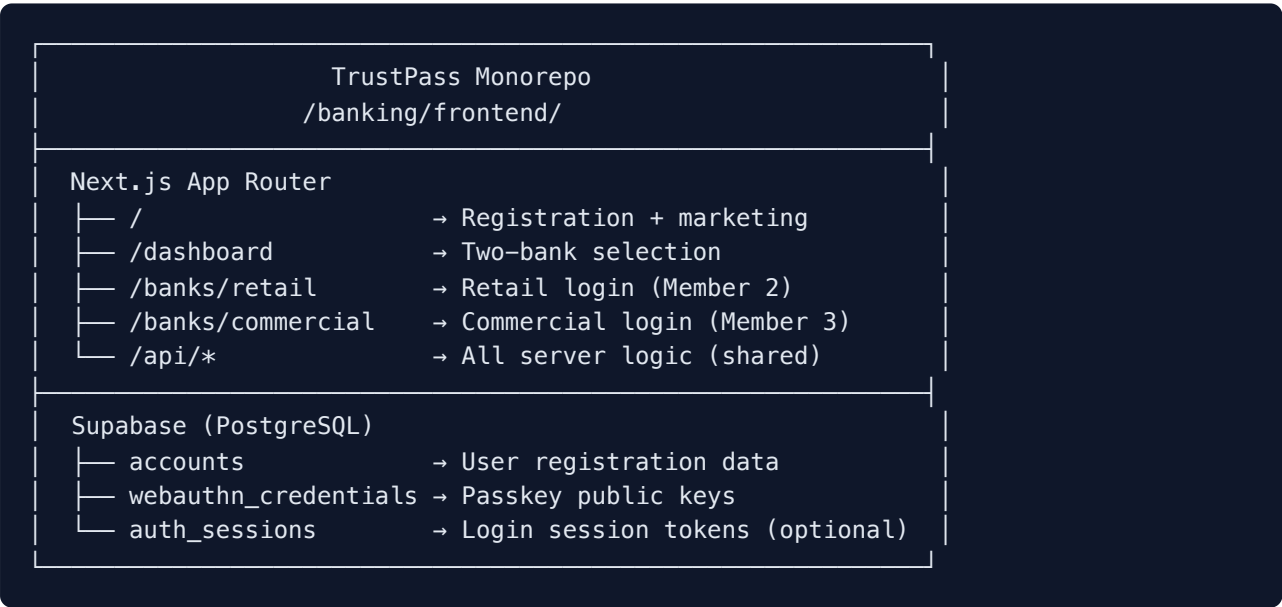
Layer	Technology	Why
Frontend	React, Next.js, HTML/CSS/JS	SSR, API routes in same repo, fast integration
Styling	Tailwind CSS, shadcn/ui	Consistent design system
Backend	Next.js API Routes (Route Handlers)	No separate Express server to merge
Database	Supabase (PostgreSQL)	SQL requirement met, free tier, RLS
Auth	WebAuthn / FIDO2 via <code>@simplewebauthn/server</code>	Industry standard for passkeys
Optional	Express.js micro-endpoint	Only if WebAuthn ceremony needs it — avoid unless necessary

Do NOT split into React frontend + Express backend + separate API repo. That caused your last integration failure.

3. Why Your Last Project Failed to Integrate

What went wrong	What we do instead
Each person got separate AI prompts for frontend/backend/API	One shared repo, one <code>README</code> , one <code>.env.example</code>
Different folder structures	Fixed structure in Section 10 — no one creates their own
API contracts defined late	Contracts in Section 13 defined before coding
No daily merges	Merge to <code>main</code> every day by 6 PM
Layer-based ownership (frontend person, backend person)	Feature-based ownership (retail auth, commercial auth, etc.)
Different env variable names	One <code>.env.example</code> — Integration Lead owns it
No one responsible for “does it run together?”	Member 1 is Integration Lead — non-negotiable

4. Architecture Decision — Single Integrated App



Rule: All HTTP endpoints live under `frontend/src/app/api/` . No separate Express app unless Integration Lead approves it in writing.

5. Team Structure & Role Assignments

We assign by **vertical feature slice**, not by layer.

Member	Role Title	Owns	Does NOT own
Member 1	Integration Lead & Database Owner	Repo, Supabase, env, API contracts, PR merges, schema migrations	UI styling details
Member 2	Retail Auth Engineer	/banks/retail , biometric WebAuthn register + login	Commercial portal
Member 3	Commercial Auth Engineer	/banks/commercial , hardware key WebAuthn register + login	Retail portal
Member 4	UI/UX & Design Lead	Shared components, responsive layout, animations, accessibility, demo slides	Database schema changes (proposes, M1 approves)

Collaboration Matrix

Task	M1	M2	M3	M4
Supabase setup	Lead	Support	Support	—
API route scaffolding	Lead	Contribute	Contribute	—
WebAuthn server logic	Review	Lead (Retail)	Lead (Commercial)	—
Dashboard bank card → portal navigation	Review	Wire retail	Wire commercial	Lead UI
Shared UI components	—	Use	Use	Lead
Daily merge to main	Lead	PR	PR	PR
Demo script	Co-write	Co-write	Co-write	Lead

6. Member 1 — Integration Lead & Database Owner

Responsibilities

- Own the GitHub repo and branch protection on `main`
- Create and maintain Supabase project
- Execute and version-control all SQL migrations

- Define and enforce API request/response contracts
- Review and merge all PRs daily
- Keep `.env.example` up to date
- Run `npm run build` before every merge
- Resolve merge conflicts — final authority

Setup Steps (Day 1 — Must complete before others start coding)

Step 1: Repository setup

```
cd banking/frontend
git init # if not already
git remote add origin <your-github-repo-url>
git checkout -b main
git push -u origin main
```

Step 2: Supabase project

1. Go to supabase.com → New Project
2. Name: `trustpass-banking`
3. Region: closest to demo location
4. Save database password securely (1Password / shared doc)

Step 3: Run base schema

1. Open Supabase → SQL Editor
2. Paste contents of `frontend/schema.sql` → Run
3. Verify `accounts` table exists in Table Editor

Step 4: Add WebAuthn schema (Member 1 creates, others use)

```
-- Run in Supabase SQL Editor (Member 1)
create table if not exists public.webauthn_credentials (
  id uuid default gen_random_uuid() primary key,
  account_id uuid not null references public.accounts(id) on delete cascade,
  credential_id text not null unique,
  public_key text not null,
  counter bigint default 0,
  device_type text not null check (device_type in ('retail', 'commercial')),
  transports text[],
  created_at timestamptz default now() not null
);

alter table public.webauthn_credentials enable row level security;

create policy "Allow credential insert" on public.webauthn_credentials
  for insert with check (true);

create policy "Allow credential read" on public.webauthn_credentials
  for select using (true);

create index credentials_account_id_idx on public.webauthn_credentials(account_id);
```

Step 5: Environment file

Create `frontend/.env.local` (never commit) and update `frontend/.env.example` :

```
NEXT_PUBLIC_SUPABASE_URL=https://xxxxx.supabase.co
NEXT_PUBLIC_SUPABASE_ANON_KEY=eyJ...
WEBAUTHN_RP_ID=localhost
WEBAUTHN_RP_NAME=TrustPass Bank
WEBAUTHN_ORIGIN=http://localhost:3000
```

Share keys with team via secure channel (not WhatsApp screenshots).

Step 6: Install WebAuthn packages

```
cd frontend
npm install @simplewebauthn/server @simplewebauthn/browser
```

Step 7: Create shared API route stubs

Create empty route handlers so M2 and M3 can fill them in without file conflicts:

Route	Purpose	Owner fills in
POST /api/webauthn/register/options	Generate registration challenge	M2 or M3
POST /api/webauthn/register/verify	Verify registration response	M2 or M3
POST /api/webauthn/login/options	Generate login challenge	M2 or M3
POST /api/webauthn/login/verify	Verify login response	M2 or M3

File layout:

```
frontend/src/app/api/webauthn/
├── register/
│   ├── options/route.ts
│   └── verify/route.ts
└── login/
    ├── options/route.ts
    └── verify/route.ts
```

Step 8: Branch protection

On GitHub: Settings → Branches → Protect `main` : - Require PR before merge - Require 1 approval (Integration Lead)

Deliverables

- ☐ Supabase project live with both tables
- ☐ `.env.example` committed
- ☐ API stub routes committed
- ☐ `docs/API-CONTRACTS.md` written (see Section 13)
- ☐ All 4 members can run `npm run dev` successfully

Files Member 1 owns exclusively

```
frontend/schema.sql
frontend/.env.example
frontend/src/lib/supabase.ts
frontend/src/app/api/register/route.ts
docs/
```

7. Member 2 — Retail Bank Passkey Authentication

Responsibilities

- Build `/banks/retail` login page
- Implement real WebAuthn registration for retail (biometric passkey)
- Implement real WebAuthn login for retail
- Wire dashboard “Retail Bank” card → `/banks/retail`
- Store credentials with `device_type = 'retail'`

Setup Steps

Step 1: Clone and run (Day 1)

```
git clone <repo-url>
cd banking/frontend
cp .env.example .env.local
# Ask M1 for real Supabase keys
npm install
npm run dev
# Open http://localhost:3000 — confirm registration works
```

Step 2: Study WebAuthn flow

Registration:

Browser → GET/POST options (server generates challenge)
Browser → `navigator.credentials.create()` (biometric prompt)
Browser → POST verify (server validates + stores public key)

Login:

Browser → POST login/options (server generates challenge)
Browser → `navigator.credentials.get()` (biometric prompt)
Browser → POST login/verify (server validates signature)

Step 3: Create retail page

```
frontend/src/app/banks/retail/page.tsx      ← Login UI
frontend/src/lib/webauthn/retail-client.ts   ← Browser-side helpers
frontend/src/app/api/webauthn/register/options/route.ts ← Add retail logic
frontend/src/app/api/webauthn/login/options/route.ts  ← Add retail logic
```

Step 4: Implement registration (after account creation)

Modify `PasskeyModal.tsx` to call real WebAuthn instead of `setTimeout` simulation: 1. Call `POST /api/webauthn/register/options` with `{ accountId, deviceType: 'retail' }` 2. Call

`startRegistration()` from `@simplewebauthn/browser` 3. Call `POST /api/webauthn/register/verify` with the result 4. On success → redirect to `/dashboard`

Step 5: Implement retail login page

UI elements: - Email or Customer ID input - "Sign in with Passkey" button - Fingerprint icon + biometric prompt - Error states (no passkey found, verification failed) - Success → retail account overview page (simple placeholder OK)

Step 6: Authenticator selection

For retail, restrict to platform authenticators:

```
authenticatorSelection: {  
  authenticatorAttachment: 'platform', // Touch ID, Face ID, Windows Hello  
  residentKey: 'preferred',  
  userVerification: 'required',  
}
```

Deliverables

- ☐ Real passkey registration in `PasskeyModal` (retail path)
- ☐ `/banks/retail` login page functional
- ☐ Credentials stored in `webauthn_credentials` with `device_type='retail'`
- ☐ Dashboard retail card navigates to `/banks/retail`
- ☐ Works on Chrome + Safari (test both)

Files Member 2 owns

```
frontend/src/app/banks/retail/**  
frontend/src/lib/webauthn/retail-client.ts  
frontend/src/app/api/webauthn/**/route.ts (shared – coordinate with M3 via PRs)  
frontend/src/components/PasskeyModal.tsx (retail WebAuthn integration)
```

Integration checkpoints with other members

- **M1:** Schema for `webauthn_credentials` must exist before Day 2
 - **M3:** Split API routes by `deviceType` param — do not duplicate files
 - **M4:** Use shared `Button`, `Card`, `Input` components — do not create new ones
-

8. Member 3 — Commercial Bank Hardware Key Authentication

Responsibilities

- Build `/banks/commercial` login page
- Implement WebAuthn registration for commercial (cross-platform / hardware key)
- Implement WebAuthn login for commercial
- Wire dashboard “Commercial Bank” card → `/banks/commercial`
- Store credentials with `device_type = 'commercial'`

Setup Steps

Step 1: Same clone/setup as Member 2 (Day 1)

Step 2: Understand hardware key differences

Commercial bank uses **cross-platform authenticators** (YubiKey, USB security keys):

```
authenticatorSelection: {  
  authenticatorAttachment: 'cross-platform', // USB/NFC hardware keys  
  residentKey: 'preferred',  
  userVerification: 'discouraged', // Some YubiKeys don't support UV  
}
```

Step 3: Create commercial page

```
frontend/src/app/banks/commercial/page.tsx  
frontend/src/lib/webauthn/commercial-client.ts
```

Step 4: Commercial registration flow

Option A: Separate enrollment step on commercial portal (user registers hardware key when first visiting commercial bank)

Option B: During initial PasskeyModal, ask “Register hardware key for commercial access?” — recommended for demo clarity

Step 5: Commercial login page UI

- Professional/enterprise aesthetic (dark accents, building icons)
- “Insert your security key” instruction panel
- USB key icon animation while waiting
- Support WebAuthn `get()` with `allowCredentials`

- Show last 4 chars of Customer ID on success

Step 6: Test with hardware

- Borrow a YubiKey if available
- Fallback: Chrome WebAuthn virtual authenticator (DevTools → More Tools → WebAuthn)
 - Create virtual authenticator with `ctap2`, `usb`, `nfc` transports

Deliverables

- ☐ `/banks/commercial` login page functional
- ☐ Hardware key registration flow
- ☐ Credentials stored with `device_type='commercial'`
- ☐ Dashboard commercial card navigates to `/banks/commercial`
- ☐ Tested with virtual or physical authenticator

Files Member 3 owns

```
frontend/src/app/banks/commercial/**
frontend/src/lib/webauthn/commercial-client.ts
frontend/src/app/api/webauthn/**/route.ts (shared – use deviceType param)
```

Coordination with Member 2

Both M2 and M3 use the **same 4 API routes**. Differentiate by request body:

```
{ "email": "user@example.com", "deviceType": "retail" }
{ "email": "user@example.com", "deviceType": "commercial" }
```

Never create separate `/api/retail/` and `/api/commercial/` folders.

9. Member 4 — UI/UX, Design System & Demo Polish

Responsibilities

- Own visual consistency across all pages
- Ensure responsive design (mobile, tablet, desktop)
- Improve dashboard, bank portals, and registration UX
- Accessibility (ARIA labels, keyboard navigation, color contrast)
- Loading states, error states, empty states for all flows
- Demo presentation slides and script

- Favicon, page titles, meta tags

Setup Steps

Step 1: Clone and audit UI (Day 1)

```
git clone <repo-url>
cd banking/frontend
npm install && npm run dev
```

Open every page. Screenshot inconsistencies. Create a Figma or shared doc with: - Color palette (already in `globals.css`) - Typography scale - Spacing rules - Component inventory

Step 2: Component inventory

Existing shared components (extend, don't duplicate):

```
frontend/src/components/ui/button.tsx
frontend/src/components/ui/card.tsx
frontend/src/components/ui/input.tsx
frontend/src/components/ui/dialog.tsx
```

Create if missing:

```
frontend/src/components/ui/badge.tsx
frontend/src/components/ui/spinner.tsx
frontend/src/components/ui/toast.tsx (or use sonner)
frontend/src/components/layout/PageHeader.tsx
frontend/src/components/layout/PageBackground.tsx
```

Step 3: Responsive testing matrix

Test every page at: - 375px (iPhone SE) - 768px (iPad) - 1280px (Laptop) - 1920px (Desktop)

Use Chrome DevTools device toolbar. Screenshot and file issues as GitHub Issues assigned to page owner.

Step 4: Wire dashboard navigation

Update `frontend/src/app/dashboard/page.tsx` : - Retail card click →

`router.push('/banks/retail')` - Commercial card click → `router.push('/banks/commercial')`

- Remove "coming soon" placeholder once M2/M3 pages exist

Step 5: Page-specific polish

Page	Polish tasks
/ (Registration)	Form validation animations, success confetti
/dashboard	Bank card hover glow, selected state
/banks/retail	Biometric scan animation, friendly copy
/banks/commercial	Enterprise dark theme, key insertion guide
All pages	Consistent header/footer, loading skeletons

Step 6: Accessibility checklist

- ☐ All inputs have `<label>` or `aria-label`
- ☐ Focus visible on keyboard tab
- ☐ Color contrast ratio $\geq 4.5:1$
- ☐ Error messages linked via `aria-describedby`
- ☐ Modals trap focus

Step 7: Demo assets

Create `docs/demo/` folder: - `DEMO-SCRIPT.md` — step-by-step demo flow - `SLIDES.pdf` — 5-6 slides (problem, solution, architecture, demo, team) - Screen recording backup (OBS, 1080p)

Deliverables

- ☐ All pages responsive on 4 breakpoints
- ☐ Shared layout components used everywhere
- ☐ Dashboard wired to both bank portals
- ☐ Demo script and slides ready
- ☐ Lighthouse accessibility score ≥ 90

Files Member 4 owns

```
frontend/src/components/ui/**
frontend/src/components/layout/**
frontend/src/app/globals.css (coordinate changes via PR)
frontend/src/app/dashboard/page.tsx (navigation wiring)
docs/demo/**
public/favicon.ico
```

10. Shared Repository Structure

11. Git Workflow & Integration Rules

Branch naming

```
feature/retail-login      (Member 2)
feature/commercial-login  (Member 3)
feature/ui-polish-dashboard (Member 4)
chore/supabase-schema-v2 (Member 1)
fix/register-validation   (anyone)
```

Daily workflow (every member, every day)

```
1. git pull origin main           ← Start of day
2. git checkout -b feature/your-task ← New branch
3. Code your piece (only your owned files)
4. npm run build                  ← Must pass locally
5. git push + open PR to main
6. Tag Integration Lead for review
7. Merge before 6 PM              ← End of day rule
```

PR rules

- **Max PR size:** 400 lines changed (split if larger)
- **PR description must include:** What changed, how to test, screenshot/video
- **No direct commits to `main`** except Integration Lead hotfixes
- **Conflicts:** Person who causes conflict resolves it same day

Communication

- **Daily standup:** 15 min on Google Meet / Discord
 - What I did yesterday
 - What I'm doing today
 - Any blockers
 - **Shared Discord/Slack channel:** `#trustpass-dev`
 - **Blockers > 2 hours:** Message Integration Lead immediately
-

12. Environment & Tooling Setup (All Members)

Required software

Tool	Version	Install
Node.js	20 LTS	nodejs.org
Git	Latest	git-scm.com
VS Code or Cursor	Latest	IDE
Chrome	Latest	For WebAuthn testing
GitHub account	—	Repo access

First-time setup (all 4 members — Day 1)

```
# 1. Clone
git clone <repo-url>
cd banking/frontend

# 2. Install dependencies
npm install

# 3. Environment (get keys from Member 1)
cp .env.example .env.local
# Edit .env.local with real Supabase keys

# 4. Verify app runs
npm run dev
# → http://localhost:3000 should show registration page

# 5. Verify build
npm run build
# → Must complete with no errors

# 6. Verify Supabase connection
# Register a test account → check Supabase Table Editor → accounts row appears
```

VS Code extensions (recommended)

- ESLint
 - Tailwind CSS IntelliSense
 - Prettier
 - GitLens
-

13. Database Schema & API Contracts

Member 1 creates `docs/API-CONTRACTS.md` with these specs:

POST `/api/register`

```
// Request
{ "fullName": "John Doe", "email": "john@mail.com", "customerId": "TP847192",
  "phoneNumber": "+15550000000" }

// Response 201
{ "success": true, "account": { "id": "uuid", "full_name": "...", "email": "...",
  "customer_id": "...", "created_at": "..." } }

// Response 400
{ "error": "An account with this email address is already registered." }
```

POST `/api/webauthn/register/options`

```
// Request
{ "accountId": "uuid", "deviceType": "retail" | "commercial" }

// Response 200
{ "options": { /* PublicKeyCredentialCreationOptionsJSON */ } }
```

POST `/api/webauthn/register/verify`

```
// Request
{ "accountId": "uuid", "deviceType": "retail" | "commercial", "response": { /*
  RegistrationResponseJSON */ } }

// Response 200
{ "verified": true, "credentialId": "..." }
```

POST `/api/webauthn/login/options`

```
// Request
{ "email": "john@mail.com", "deviceType": "retail" | "commercial" }

// Response 200
{ "options": { /* PublicKeyCredentialRequestOptionsJSON */ } }
```

POST /api/webauthn/login/verify

```
// Request
{ "email": "john@mail.com", "deviceType": "retail" | "commercial", "response": { /*
  AuthenticationResponseJSON */ } }

// Response 200
{ "verified": true, "account": { "id": "uuid", "full_name": "...", "customer_id":
  "..." } }
```

Rule: If any member needs to change a contract, they open a PR updating API-CONTRACTS.md first. Integration Lead approves. Then implementation follows.

14. 4-Week Sprint Timeline

Week 1 — Foundation (Integration First)

Day	Member 1	Member 2	Member 3	Member 4
Mon	Supabase + schema + env + API stubs	Clone, run app, read WebAuthn docs	Clone, run app, read WebAuthn docs	UI audit, design tokens doc
Tue	API-CONTRACTS.md + PR reviews	PasskeyModal → real WebAuthn register (retail)	Research hardware key flow	Shared layout components
Wed	webauthn_credentials table + RLS	Register/options + register/verify API (retail)	Commercial-client.ts scaffold	Dashboard → portal navigation
Thu	Merge all PRs, fix integration bugs	Retail register end-to-end test	Commercial register scaffold	Responsive fixes on registration
Fri	Integration test: register → passkey → dashboard	Bug fixes	Bug fixes	Demo wireframes

Week 1 exit criteria: User can register, enroll a real biometric passkey, land on dashboard.

Week 2 — Bank Portals

Day	Member 1	Member 2	Member 3	Member 4
Mon	Login API stubs + session handling	/banks/retail login page UI	/banks/commercial login page UI	Retail page styling
Tue	PR reviews	Retail login WebAuthn flow	Commercial register flow	Commercial page styling
Wed	Integration testing both portals	Retail login end-to-end	Commercial login end-to-end	Loading/error states
Thu	Fix cross-portal bugs	Edge cases (no passkey, wrong device)	Virtual authenticator testing	Mobile responsive pass
Fri	Full flow test: register → dashboard → both logins	Polish	Polish	Accessibility audit

Week 2 exit criteria: Both bank portals have working passwordless login.

Week 3 — Polish & Innovation

Day	All members
Mon	Pick 2 bonus features from Section 17
Tue–Thu	Implement bonus features + UI polish
Fri	Full regression test on all browsers

Week 4 — Demo Prep

Day	All members
Mon	Fix all open bugs, run Lighthouse audits
Tue	Record demo video backup
Wed	Rehearse 5-min demo (3 runs minimum)
Thu	Final code freeze — bug fixes only
Fri	Submit + present

15. Daily Integration Checklist

Integration Lead runs this every day at 6 PM:

```
[ ] git pull origin main on a clean machine
[ ] npm install (no new dependency conflicts)
[ ] npm run build — passes
[ ] npm run dev — app loads
[ ] Register new test account — succeeds
[ ] Supabase accounts table — new row appears
[ ] Passkey enrollment — works (or simulated if WebAuthn not ready yet)
[ ] Dashboard — both bank cards visible
[ ] /banks/retail — loads without error
[ ] /banks/commercial — loads without error
[ ] No console errors on any page
[ ] All open PRs merged or have clear blockers noted
```

If any check fails → **stop new feature work**, fix integration first.

16. Judging Criteria — How We Score Points

Criterion	Our strategy	Owner
Consistency	Shared design system, one repo, one API pattern, daily merges	M4 + M1
Completion	Week-by-week exit criteria (Section 14), pre-submission checklist	M1
Innovation & Additional Features	Section 17 backlog — pick 2–3	All
UI/UX Design	Premium fintech aesthetic, animations, clear auth flows	M4
Integration	Single Next.js app, shared API contracts, daily integration test	M1
Responsiveness	Test 4 breakpoints, mobile-first bank login	M4
Originality	Custom TrustPass branding, original copy, cite WebAuthn libraries only	All

Minimum viable demo (must work on presentation day)

1. Register account with form validation
2. Enroll biometric passkey (real WebAuthn)
3. See two-bank dashboard

4. Login to Retail Bank with passkey
5. Login to Commercial Bank with hardware key (or virtual authenticator)
6. Show Supabase database with stored credentials

17. Innovation & Bonus Features Backlog

Pick **2–3** after core flows work. Assign one owner each.

Feature	Impact	Effort	Suggested owner
Session timeout + re-auth prompt	Security	Medium	M1
“Remember this device” toggle	UX	Low	M2
Login attempt audit log in Supabase	Security	Medium	M1
QR code cross-device auth	Innovation	High	M2
Dark mode toggle	UI/UX	Low	M4
Account recovery via email OTP	Completeness	High	M1 + M2
Animated security score dashboard	UI/UX	Medium	M4
Rate limiting on login API	Security	Medium	M1
Multi-language support (EN + Hindi)	UX	Medium	M4
Passkey management page (view/revoke keys)	Completeness	Medium	M2 + M3

18. Demo Script (5 Minutes)

Member 4 leads presentation. Others operate the demo.

Time	Action	Speaker
0:00–0:45	"Passwords are broken in banking. TrustPass eliminates them." Show problem slide.	M4
0:45–1:30	Architecture slide: one identity, two banks, two auth methods	M1
1:30–2:30	Live demo: Register new account → form validation → Supabase row	M2
2:30–3:15	Enroll biometric passkey → show browser security prompt	M2
3:15–3:45	Dashboard → select Retail Bank → login with passkey	M2
3:45–4:30	Back to dashboard → Commercial Bank → login with security key	M3
4:30–5:00	Show Supabase tables + Q&A	M1

Backup plan: If live WebAuthn fails, play pre-recorded video (M4 prepares).

19. Risk Register & Mitigation

Risk	Likelihood	Impact	Mitigation
WebAuthn doesn't work on demo laptop	Medium	High	Pre-record video; test on demo device Week 3
Merge conflicts break main	High	High	Daily merges, small PRs, M1 reviews
Members build duplicate components	Medium	Medium	M4 owns component library; PR review catches duplicates
Supabase free tier limits	Low	Medium	Delete test rows; one project only
No physical YubiKey available	Medium	Low	Chrome virtual authenticator
localhost WebAuthn RP ID issues	Medium	High	Set <code>WEBAUTHN_RP_ID=localhost</code> in <code>.env</code> ; deploy to Vercel if needed
Member falls behind	Medium	High	Daily standup blockers; reassign tasks immediately

20. Originality & Plagiarism Guidelines

Allowed

- Official docs: [WebAuthn Guide](#), [SimpleWebAuthn docs](#)
- shadcn/ui components (open source, MIT license)
- Supabase documentation patterns
- Your own code from this TrustPass project

Not allowed

- Copying another team's entire repo
- Using a commercial banking template without significant modification

- Submitting AI-generated code without understanding and customizing it

How to stay original

- Custom TrustPass branding (logo, colors, copy)
- Two-bank architecture is your unique angle
- Write your own API route logic (use SimpleWebAuthn library, not copy-paste tutorials verbatim)
- Document your architecture decisions in README
- Git history shows incremental team contributions

README attribution section (add before submission)

```
## Attributions
- UI components: shadcn/ui (MIT)
- WebAuthn: @simplewebauthn/server, @simplewebauthn/browser (MIT)
- Database: Supabase (PostgreSQL)
- Icons: lucide-react (ISC)
```

21. Pre-Submission Checklist

Code quality

- ☐ `npm run build` passes with zero errors
- ☐ No `console.log` left in production code
- ☐ `.env.local` is NOT committed (check `.gitignore`)
- ☐ All API routes return proper HTTP status codes
- ☐ Error messages are user-friendly

Functionality

- ☐ Registration with validation works
- ☐ Duplicate email/Customer ID shows error
- ☐ Real WebAuthn passkey registration works
- ☐ Dashboard shows both banks
- ☐ Retail login with biometric works
- ☐ Commercial login with hardware/virtual key works
- ☐ Data visible in Supabase tables

UI/UX

- ☐ Responsive on mobile (375px), tablet (768px), desktop (1280px)
- ☐ Consistent header/footer on all pages
- ☐ Loading states on all async actions
- ☐ Error states on all forms
- ☐ Lighthouse Performance ≥ 80 , Accessibility ≥ 90

Documentation

- ☐ README.md with setup instructions
- ☐ `.env.example` with all required variables
- ☐ `schema.sql` up to date
- ☐ `docs/API-CONTRACTS.md` matches actual API
- ☐ Demo script rehearsed 3+ times

Submission

- ☐ GitHub repo link ready (public or judges have access)
- ☐ Deployed URL (Vercel recommended — free tier)
- ☐ Demo video uploaded as backup
- ☐ Team member contributions visible in Git history

Quick Reference Card

```
REPO:      banking/frontend/  (single Next.js app)
DATABASE:   Supabase PostgreSQL
AUTH:       WebAuthn via @simplewebauthn
MERGE:      Daily to main by 6 PM
PR SIZE:    Max 400 lines
CONTRACTS:  docs/API-CONTRACTS.md (change before code)
INTEGRATION: Member 1 runs checklist daily
DEMO:       Register → Passkey → Dashboard → 2 Logins
```

TrustPass Team — Passwordless Authentication for Bank Logins
Document maintained by Integration Lead (Member 1)